

پاسخ سوالات پروژه موس هوایی (Air Mouse)

سوال ۱ — از درخواست خواندن داده از سنسور تا دریافت آن، چه اتفاقاتی در سطح سیستم عامل می افتد؟

وقتی برنامه‌ی اندرویدی با `SensorManager.registerListener()` یک سنسور را ثبت می‌کند، زنجیره‌ی زیر در سیستم عامل اتفاق می‌افتد:

1. لایه‌ی **HAL (Hardware Abstraction Layer)**: درخواست از طریق `SensorService` (یک سرویس سیستمی در فضای کاربری) به `HAL` سنسور ارسال می‌شود. `HAL` با درایور کرنل ارتباط برقرار می‌کند.
2. **درایور کرنل و وقفه (Interrupt)**: سخت‌افزار سنسور (مثلاً `IMU`) داده را آماده می‌کند و یک وقفه‌ی سخت‌افزاری به `CPU` ارسال می‌کند.
3. **Context Switch**: کرنل وقفه را می‌پذیرد، `thread` فعلی را متوقف کرده و `ISR (Interrupt Service Routine)` درایور را اجرا می‌کند.
4. **انتقال داده به فضای کاربری**: داده از بافر کرنل از طریق مکانیزم `Binder IPC` به `SensorService` منتقل می‌شود.
5. **ارسال به اپلیکیشن**: `SensorService` داده را از طریق یک صف رویداد (`event queue`) روی یک `thread` جداگانه به `onSensorChanged()` (اپلیکیشن تحویل می‌دهد).

در خروجی Perfetto می‌توان این مراحل را به شکل زیر مشاهده کرد:

- ردیف `SensorService` نشان‌دهنده‌ی پردازش در سرویس سیستمی است
- ردیف `binder` نشان‌دهنده‌ی `IPC` بین `SensorService` و اپلیکیشن است
- ردیف `thread` اپلیکیشن لحظه‌ی فراخوانی `onSensorChanged` را نشان می‌دهد

سوال ۲ — کارکرد سنسورها، دلیل خطا، و تأثیر ترکیب آن‌ها

ژیروسکوپ (Gyroscope): سرعت زاویه‌ای حول سه محور را بر حسب `rad/s` اندازه می‌گیرد. با انتگرال‌گیری از این مقادیر، می‌توان زاویه‌ی دوران را به دست آورد. مشکل اصلی آن **drift** (انباشت خطا در طول زمان) است؛ چون نویز کوچک پس از انتگرال‌گیری تجمع می‌شود.

شتاب‌سنج (Accelerometer): شتاب خطی (شامل گرانش) را اندازه می‌گیرد. می‌توان از آن برای تخمین زاویه‌ی ثابت نسبت به بردار گرانش استفاده کرد. مشکل اصلی آن **نویز بالا** در حرکات سریع و ناتوانی در تشخیص دوران حول محور عمودی (`yaw`) است.

مغناطیس‌سنج (Magnetometer): میدان مغناطیسی زمین را اندازه می‌گیرد و برای تخمین زاویه‌ی `yaw` (راستای شمال) استفاده می‌شود. مشکل آن **تداخل با میدان‌های مغناطیسی محیطی** (آهن، الکترومغناطیس) و نیاز به کالیبراسیون مکرر است.

چرا ترکیب کمک می‌کند؟ هر سنسور در فرکانس‌های متفاوتی قابل اعتماد است:

- ژیروسکوپ در کوتاه‌مدت دقیق است اما در بلندمدت **drift** دارد
- شتاب‌سنج و مغناطیس‌سنج در بلندمدت قابل اعتمادند اما در لحظه نویز دارند

فیلترهایی مانند Madgwick این دو را ترکیب می‌کنند: ژيروسکوپ برای به‌روزرسانی لحظه‌ای و شتاب‌سنج/مغناطیس‌سنج برای تصحیح drift.

سوال ۳ — مقایسه‌ی زمان بین دو نمونه‌ی متوالی در Perfetto با دوره‌ی نمونه‌برداری کد

در اندروید هنگام ثبت سنسور، نرخ نمونه‌برداری با فلگ‌هایی مثل `SENSOR_DELAY_FASTEST` یا برحسب میکروثانیه تعیین می‌شود. `SENSOR_DELAY_GAME`

- در کد: مثلاً اگر سنسور روی `20ms` (معادل `50Hz`) تنظیم شده باشد، انتظار می‌رود فواصل زمانی دقیقاً همین مقدار باشد.
- در Perfetto: در عمل زمان بین دو رخداد متوالی (`Timestamp`های ثبت شده در تریس) دقیقاً ثابت نیست و پدیده `Jitter` (نوسان زمانی) دیده می‌شود. دلیل این تفاوت، ماهیت بی‌درنگ نرم (`Soft Real-Time`) بودن اندروید است. اولویت‌بندی زمان‌بند هسته (`Scheduler`)، وقوع وقفه‌های سخت‌افزاری دیگر، فرآیند کارهای پس‌زمینه (`Garbage Collection`) و تأخیر در پاسخ‌گویی به وقفه (`Interrupt Latency`) باعث می‌شود که فواصل داده‌ها در Perfetto کمی نوسان داشته باشد و دقیقاً منطبق بر دوره ایده‌آل تنظیم‌شده در کد نباشد.

سوال ۴ — آیا تعارض (`Conflict`) بین فعالیت‌های حساس به زمان و پردازش‌های سنگین رخ می‌دهد؟

بله، این تعارض وجود دارد. در Perfetto می‌توان مشاهده کرد:

- **Thread توجیه:** هرچند سیستم‌عامل اندروید تِرد مربوط به رندر گرافیکی (`RenderThread`) و تِرد سنسورها را در لایه‌های متفاوتی اجرا می‌کند، اما اگر این تِردها روی یک هسته مشترک از پردازنده زمان‌بندی شوند، یا هر دو نیاز به فراخوانی‌های سیستمی مکرر (`System Calls`) داشته باشند، یک تِرد ممکن است منتظر آزاد شدن هسته یا قفل‌های مشترک سیستم‌عامل (`Mutex Locks` در لایه سرویس‌ها) بماند.
- اگر پردازش رندر گرافیکی به قدری سنگین باشد که تمام ظرفیت سی‌پی‌یو را درگیر کند یا در فراخوانی‌های سیستمی مانند قفل کردن بافر گرافیکی بلاک شود، زمان‌بند لینوکس ناچار است بین فعال‌سازی تِرد سنسور و تِرد گرافیکی، دست به جابه‌جایی تعویض خط (`Context Switch`) بزند. در Perfetto این پدیده به صورت طولانی شدن زمان وضعیت `Runnable` در تِرد سنسور یا تأخیر در زدن کانال ارتباطی سوکت (`UDP`) به وضوح قابل توجیه است.
- زمان‌بند لینوکس با سیاست `CFS (Completely Fair Scheduler)` این `thread`ها را اولویت‌بندی می‌کند

توجیه: در Perfetto در ردیف CPU می‌توان دید که گاهی `thread` سنسور منتظر می‌ماند تا رندر UI تمام شود. راحمل:

استفاده از `SENSOR_DELAY_FASTEST` و پردازش سنسور روی یک `thread` با اولویت بالاتر (مثلاً با

`Process.THREAD_PRIORITY_URGENT_AUDIO`).

Thread: SensorsSubHal | State: Waiting for CPU (Runnable) | Total delay: 346.16 ms

Thread: RenderThread | State: Waiting for CPU (Runnable) | Total delay: 225.08 ms

Thread: SensorService | State: Waiting for CPU (Runnable) | Total delay: 118.71 ms

Thread: SensorHubManage | State: Waiting for CPU (Runnable) | Total delay: 2.52 ms

Thread: RenderThread | State: Blocked on syscall/IO | Total delay: 0.64 ms

سوال ۵ — تفاوت wake-up و non-wake-up سنسور

ویژگی	Wake-up	Non-wake-up
رفتار در sleep	پردازنده را بیدار می‌کند	داده را در بافر نگه می‌دارد
مصرف باتری	بیشتر	کمتر
تأخیر داده	کمتر	بیشتر (تا خواندن بافر)
کاربرد مناسب	تشخیص رویدادهای فوری (مثل کلیک)	نمونه‌برداری پیوسته در پس‌زمینه

در این پروژه: برای کنترل ماوس که نیاز به پاسخ‌دهی بلادرنگ دارد، **wake-up** مناسب‌تر است؛ اما مصرف باتری بیشتری دارد. برای ضبط داده در پس‌زمینه non-wake-up کافی است.

سوال ۶ — میانگین زمان اجرای تابع فیلتر روی CPU

با تعریف tracepoint سفارشی در کد اندروید:

```
Trace.beginSection("MadgwickFilter")
madgwickUpdate(gx, gy, gz, ax, ay, az, mx, my, mz)
Trace.endSection()
```

سپس در Perfetto با کوئری SQL:

```
SELECT AVG(dur)/1e6 as avg_ms
FROM slice
WHERE name = 'MadgwickFilter'
```

زمان اجرای فیلتر Madgwick روی یک گوشی متوسط بین ۰.۱ تا ۰.۵ میلی‌ثانیه است که نسبت به دوره‌ی نمونه‌برداری (۲۰ms) بسیار کوچک است.

سوال ۷ — بیشترین مصرف پردازشی مربوط به کدام سنسور است؟

با کوئری Perfetto روی رویدادهای سنسور، معمولاً ژيروسکوپ به دلیل نرخ نمونه‌برداری بالاتر (تا ۸۰۰Hz) در برخی دستگاه‌ها و نیاز به انتگرال‌گیری، بیشترین بار پردازشی را دارد. مغناطیس‌سنج کمترین نرخ نمونه‌برداری را دارد و کمترین بار را ایجاد می‌کند.

سوال ۸ — تأثیر نرخ نمونه‌برداری سنسورها بر سایر پردازش‌های سیستمی

وقتی نرخ نمونه‌برداری (Sampling Rate) سنسورها را روی بالاترین حد یعنی SENSOR_DELAY_FASTEST (که بسته به سخت‌افزار فرکانسی بین ۱۰۰ تا بیش از ۲۰۰ هرتز دارد) تنظیم می‌کنید، بار پردازشی سنگینی به کل سیستم تحمیل می‌شود. این تأثیرات را می‌توان در سه لایه بررسی کرد:

۱. **لایه هسته و افزایش تعویض خط (Context Switch Overhead):** هر داده‌ای که توسط سخت‌افزار سنسور تولید می‌شود، یک وقفه (Hardware Interrupt) ایجاد می‌کند. هسته سیستم‌عامل (Kernel) باید اجرای جاری خود را متوقف کرده، روال خدمت‌رسانی به وقفه (ISR) را اجرا کند و داده را به لایه درایور بفرستد. بالا رفتن فرکانس به معنای جابه‌جایی مکرر و پی‌درپی پردازنده بین حالت کاربر (User Space) و حالت هسته (Kernel Space) است که بخش زیادی از توان پردازشی CPU را صرف این «بار اضافی» یا *Overhead* می‌کند.

۲. **اشباع کردن سیستم‌سرور (System Server Flooding):** در اندروید، سرویس SensorService در داخل فرآیند حیاتی system_server اجرا می‌شود. وقتی سنسور با نرخ بالا داده تولید می‌کند، این سرویس باید به طور مداوم داده‌ها را از لایه HAL دریافت کرده و از طریق کانال‌های ارتباطی (مانند لوپرها و بایت‌بافرهای IPC) به اپلیکیشن شما دیسپچ کند. این کار باعث درگیر شدن یکی از هسته‌های CPU توسط system_server شده و ممکن است پاسخ‌دهی سیستم به دیگر کارهای حیاتی (مثل مدیریت اعلانات، ورودی‌های لمسی صفحه و مدیریت حافظه) را با تأخیر مواجه کند.

۳. **پدیده Jank یا افت فریم در لایه گرافیک:** اگر اپلیکیشن شما یا سایر برنامه‌های در حال اجرا بخواهند کارهای گرافیکی انجام دهند، به دلیل رقابت بر سر تصاحب هسته‌های پردازنده (CPU Contention)، ترد رندرینگ (RenderThread) ممکن است نتواند در زمان مقرر (مثلاً هر ۱۶.۶ میلی‌ثانیه برای نمایشگر ۶۰ هرتز) کارهای خود را به پایان برساند. در نتیجه، کاربر در محیط اندروید شاهد افت فریم، لگ یا به اصطلاح *Jank* خواهد بود.

سوال ۹ — زمان کلی از آماده شدن داده سنسور تا حرکت مکان‌نما

این زنجیره شامل مراحل زیر است:

مرحله	زمان تقریبی
سنسور تا onSensorChanged	۱ تا ۵ ms
پردازش فیلتر	۰.۱ تا ۰.۵ ms
بسته‌بندی و ارسال UDP	۱ تا ۳ ms
شبکه‌ی محلی (WiFi)	۱ تا ۱۰ ms
پردازش در Python	۰.۵ تا ۲ ms
PyAutoGUI و سیستم‌عامل لپ‌تاپ	۱ تا ۵ ms
جمع کل	~۵ تا ۲۵ ms

این تأخیر (latency) در شرایط معمولی قابل قبول است، اما در شبکه‌های شلوغ یا WiFi ضعیف می‌تواند به ۵۰ms یا بیشتر برسد.

سوال ۱۰ — تفکیک threadها و رویدادهای سنسور

در Perfetto thread های مختلف قابل تشخیص هستند:

- **Main Thread (UI Thread):** رندر رابط کاربری، مدیریت lifecycle. رویدادهای onSensorChanged نباید روی این thread اجرا شوند.
- **SensorEventListener Thread:** thread که توسط HandlerThread یا Executor ایجاد می‌شود و callback سنسور را دریافت می‌کند.
- **Network Thread:** thread جداگانه برای ارسال UDP (اتصال شبکه در Android روی Main Thread مجاز نیست).
- **RenderThread:** thread جداگانه‌ی سیستمی برای رندر OpenGL/Vulkan.

هر سنسور رویدادهای TYPE_SENSOR_EVENT تولید می‌کند که روی thread ثبت‌شده (Looper) تحویل داده می‌شوند. اگر callback سنسور کند باشد، **Main Thread** را بلاک نمی‌کند، اما اگر UI در callback به‌روز شود (مثلاً runOnUiThread)، می‌تواند فریم را تأخیر دهد.

Thread name: surfaceflinger | Total active time: 9885.15 ms

Thread name: RenderThread | Total active time: 5347.34 ms

Thread name: xample.airmouse | Total active time: 2185.19 ms

Thread name: HwBinder:1080_1 | Total active time: 561.87 ms

Thread name: TimerDispatch | Total active time: 282.91 ms

سوال ۱۱ — تفاوت حرکت آرام و ناگهانی بر پردازش و تأخیر

حرکت آرام:

- تغییرات کوچک در داده‌ی سنسور
- فیلتر Madgwick به راحتی همگرا می‌ماند
- تأخیر کمتر چون بسته‌های UDP حاوی تغییرات کوچک هستند و احتمال drop آنها کمتر است

حرکت ناگهانی:

- تغییرات بزرگ و سریع → ژيروسکوپ مقادیر بالا گزارش می‌دهد
- پردازش فیلتر ممکن است چند iteration بیشتر نیاز داشته باشد تا همگرا شود
- تعداد بسته‌های UDP ارسالی افزایش می‌یابد که ممکن است باعث ازدحام شبکه یا drop بسته شود
- تأخیر ماوس در حرکت ناگهانی معمولاً بیشتر است، چون سیستم‌عامل ممکن است thread سنسور را در اولویت پایین‌تری schedule کند وقتی بار پردازشی بالا می‌رود